

DOCUMENTAÇÃO TÉCNICA

Portal Institucional - Clube de Tiro “Alvo Certo”

1. ARQUITETURA DO SISTEMA

1.1 Visão Geral

O sistema foi desenvolvido utilizando uma arquitetura moderna baseada em microsserviços, com separação clara entre frontend e backend.

Stack Tecnológico:

- Frontend: React 19 + TypeScript + Tailwind CSS 4
- Backend: Node.js + Express 4 + tRPC 11
- Banco de Dados: MySQL/TiDB
- Hospedagem: Google Cloud Run
- ORM: Drizzle ORM
- Autenticação: OAuth 2.0 (Manus)

1.2 Componentes Principais

Frontend (client/):

- Aplicação React com roteamento via Wouter
- Componentes UI baseados em shadcn/ui
- Integração tRPC para comunicação com backend
- Validação de formulários com React Hook Form + Zod

Backend (server/):

- API tRPC com procedures públicas e protegidas
- Routers modulares para diferentes funcionalidades
- Middleware de autenticação OAuth
- Integração com banco de dados via Drizzle ORM

Banco de Dados:

- Tabelas: users, weapons, ammunition, courses, schedule_requests
- Relacionamentos normalizados
- Índices otimizados para consultas frequentes

2. ESTRUTURA DE DIRETÓRIOS

```
alvo-certo-portal/
├── client/                                # Frontend React
│   ├── src/
│   │   ├── pages/                        # Páginas da aplicação
│   │   │   ├── Home.tsx                 # Página inicial
│   │   │   ├── Schedule.tsx             # Formulário de agendamento
│   │   │   └── AdminDashboard.tsx       # Painel administrativo
│   │   ├── components/                  # Componentes reutilizáveis
│   │   ├── lib/                          # Utilitários e hooks
│   │   └── App.tsx                       # Componente raiz
│   └── index.html                        # Template HTML
├── server/                               # Backend Node.js
│   ├── routers/                          # Routers tRPC
│   │   ├── catalog.ts                   # Router de catálogos
│   │   ├── schedule.ts                  # Router de agendamentos
│   │   └── schedule.test.ts             # Testes
│   ├── db.ts                             # Funções de banco de dados
│   ├── routers.ts                        # Router principal
│   └── _core/                            # Código de infraestrutura
├── drizzle/                              # Schema e migrações
│   ├── schema.ts                         # Definição de tabelas
│   └── migrations/                       # Histórico de migrações
├── docs/                                 # Documentação
└── package.json                          # Dependências
```

3. SCHEMA DO BANCO DE DADOS

3.1 Tabela: users

Armazena informações de usuários autenticados.

Campo	Tipo	Descrição
id	INT	Chave primária auto-incrementada
openId	VARCHAR(64)	Identificador único do OAuth
name	TEXT	Nome do usuário
email	VARCHAR(320)	Email do usuário
loginMethod	VARCHAR(64)	Método de autenticação
role	ENUM	Papel do usuário (user, admin)
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de última atualização
lastSignedIn	TIMESTAMP	Último acesso

3.2 Tabela: weapons

Catálogo de armamentos.

Campo	Tipo	Descrição
id	INT	Chave primária
name	VARCHAR(255)	Nome do armamento
category	VARCHAR(100)	Categoria (Pistola, Revólver, Rifle, etc)
caliber	VARCHAR(50)	Calibre
manufacturer	VARCHAR(255)	Fabricante
description	TEXT	Descrição completa
specifications	TEXT	Especificações técnicas (JSON)
imageUrl	TEXT	URL da imagem
inStock	INT	Quantidade em estoque
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

3.3 Tabela: ammunition

Catálogo de munições.

Campo	Tipo	Descrição
id	INT	Chave primária
name	VARCHAR(255)	Nome da munição
caliber	VARCHAR(50)	Calibre
manufacturer	VARCHAR(255)	Fabricante
description	TEXT	Descrição
specifications	TEXT	Especificações técnicas (JSON)
imageUrl	TEXT	URL da imagem
unitsPerBox	INT	Unidades por caixa
inStock	INT	Quantidade em estoque
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

3.4 Tabela: courses

Catálogo de cursos.

Campo	Tipo	Descrição
id	INT	Chave primária
name	VARCHAR(255)	Nome do curso
description	TEXT	Descrição completa
duration	INT	Duração em horas
prerequisites	TEXT	Pré-requisitos
instructor	VARCHAR(255)	Nome do instrutor
maxParticipants	INT	Máximo de participantes
imageUrl	TEXT	URL da imagem
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

3.5 Tabela: `schedule_requests`

Solicitações de agendamento.

Campo	Tipo	Descrição
id	INT	Chave primária
fullName	VARCHAR(255)	Nome completo
email	VARCHAR(320)	Email
phone	VARCHAR(20)	Telefone
documentType	ENUM	Tipo de documento (CAC, RG)
documentNumber	VARCHAR(50)	Número do documento
requestedDate	DATE	Data solicitada
requestedTime	VARCHAR(5)	Hora solicitada (HH:MM)
numberOfPeople	INT	Número de pessoas
experience	ENUM	Nível de experiência
observations	TEXT	Observações
status	ENUM	Status (pending, approved, rejected, completed)
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data de atualização

4. ENDPOINTS DA API tRPC

4.1 Catálogos (Públicos)

catalog.weapons - GET

- Retorna lista de armamentos
- Sem autenticação necessária

catalog.ammunition - GET

- Retorna lista de munições

- Sem autenticação necessária

catalog.courses - GET

- Retorna lista de cursos
- Sem autenticação necessária

4.2 Agendamentos

schedule.create - POST (Público)

- Cria nova solicitação de agendamento
- Valida CAC/RG
- Envia notificação ao administrador

schedule.list - GET (Protegido - Admin)

- Retorna todas as solicitações
- Requer autenticação com role admin

schedule.getById - GET (Protegido - Admin)

- Retorna detalhes de uma solicitação
- Requer autenticação com role admin

schedule.updateStatus - PATCH (Protegido - Admin)

- Atualiza status de uma solicitação
- Requer autenticação com role admin

4.3 Autenticação

auth.me - GET

- Retorna dados do usuário autenticado

auth.logout - POST

- Faz logout do usuário
-

5. FLUXO DE AUTENTICAÇÃO

5.1 OAuth 2.0 Flow

1. Usuário clica em “Entrar”
2. Redirecionado para portal OAuth do Manus
3. Usuário autentica com credenciais
4. Callback para `/api/oauth/callback`
5. Session cookie criado
6. Usuário redirecionado para dashboard

5.2 Proteção de Rotas

Rotas administrativas são protegidas por middleware que verifica:

- Presença de session cookie válido
- Role do usuário (admin)
- Validade do token JWT

6. VALIDAÇÃO DE DADOS

6.1 CAC (Colecionador, Atirador e Caçador)

Formato esperado: `XXXXXX-XX`

- 6 dígitos
- Hífen
- 2 letras maiúsculas

Exemplo: `123456-AB`

6.2 RG (Registro Geral)

Formato esperado: Apenas números

- Mínimo 5 dígitos
- Máximo 12 dígitos

Exemplo: 1234567

6.3 Validação de Data

- Não permite datas no passado
 - Formato: YYYY-MM-DD
 - Validação no frontend e backend
-

7. NOTIFICAÇÕES

7.1 Sistema de Notificações

Quando uma nova solicitação é recebida, o sistema:

1. Valida os dados
2. Armazena no banco de dados
3. Envia notificação ao administrador
4. Retorna confirmação ao cliente

7.2 Conteúdo da Notificação

A notificação contém:

- Nome do solicitante
 - Data e hora do agendamento
 - Tipo e número do documento
 - Link para visualizar no painel
-

8. PERFORMANCE E OTIMIZAÇÃO

8.1 Índices de Banco de Dados

- Índice em `schedule_requests.status` para filtros rápidos
- Índice em `schedule_requests.requestedDate` para buscas por data
- Índice em `users.openId` para autenticação

8.2 Cache

- Frontend: React Query com cache automático
- Backend: Sem cache (dados sempre frescos)

8.3 Compressão

- Gzip habilitado para respostas HTTP
 - Assets minificados em produção
-

9. SEGURANÇA

9.1 HTTPS

Todas as comunicações utilizam HTTPS com certificado SSL válido.

9.2 CORS

CORS configurado para aceitar requisições apenas do domínio autorizado.

9.3 Rate Limiting

- Limite de 100 requisições por minuto por IP
- Limite de 10 agendamentos por hora por email

9.4 Validação de Entrada

- Validação no frontend com Zod
- Validação no backend com Zod
- Sanitização de strings

9.5 SQL Injection

Proteção via Drizzle ORM com prepared statements.

10. DEPLOYMENT

10.1 Build

```
pnpm build
```

Gera:

- Frontend compilado em `dist/client`
- Backend compilado em `dist/index.js`

10.2 Deploy

O sistema é deployado automaticamente no Google Cloud Run:

1. Código é enviado para repositório
2. Cloud Build detecta mudanças
3. Build é executado
4. Container é deployado
5. Tráfego é roteado para nova versão

10.3 Variáveis de Ambiente

Configuradas no Cloud Run:

- `DATABASE_URL` : Conexão MySQL
 - `JWT_SECRET` : Chave para assinar tokens
 - `VITE_APP_ID` : ID da aplicação OAuth
 - `OAUTH_SERVER_URL` : URL do servidor OAuth
-

11. MONITORAMENTO

11.1 Logs

Logs centralizados no Cloud Logging:

- Erros de aplicação
- Requisições HTTP
- Erros de banco de dados

11.2 Alertas

Alertas configurados para:

- Taxa de erro > 1%
- Tempo de resposta > 5 segundos
- Indisponibilidade do banco de dados

11.3 Métricas

Métricas coletadas:

- Requisições por segundo
 - Tempo médio de resposta
 - Taxa de erro
 - Uso de CPU e memória
-

12. MANUTENÇÃO

12.1 Backup

Backups automáticos do banco de dados:

- Diário (retenção 7 dias)
- Semanal (retenção 4 semanas)
- Mensal (retenção 12 meses)

12.2 Atualizações

Atualizações de segurança são aplicadas automaticamente.

12.3 Suporte

Para questões técnicas, contate: suporte@alvo-certo.com

Documento Preparado por: Equipe de Desenvolvimento Manus

Data de Preparação: 27 de maio de 2026

Versão: 1.0